

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Пермский национальный исследовательский политехнический
университет»**

Электротехнический факультет

Выпускающая кафедра: Информационные технологии и
автоматизированные системы (ИТАС)

Направление подготовки: 09.03.04 Программная инженерия

ОТЧЕТ

Лабораторная работа №6

«АТД. Контейнеры»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 15

Выполнил: студент группы РИС-25-2Б

Ильинов Фёдор Михайлович

Принял: Доц. Полякова О. А.

Пермь 2026

ПОСТАНОВКА ЗАДАЧИ

Постановка задачи:

Класс- контейнер СПИСОК с ключевыми значениями типа `int`.

Реализовать:

`List::operator[]` – доступа по индексу;

`List::operator()` – определение размера списка;

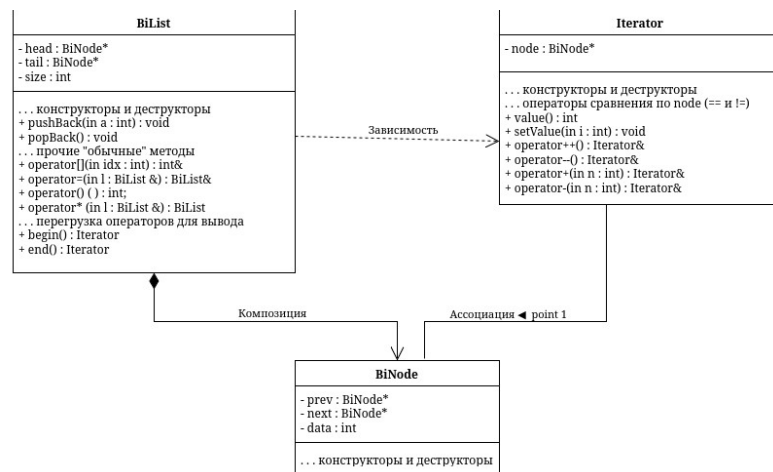
`List::operator*` – умножение элементов списков `a[i]*b[i]`;

`Iterator::operator+` - переход вправо на `n` элементов

АНАЛИЗ

1. Создадим класс `BiNode` – узел в списке.
2. Создадим класс `BiList` – двунаправленный список.
3. Создадим класс `Iterator` для перемещения по списку.

UML ДИАГРАММА



КОД Bidirectional.h

```
#ifndef ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_BI_H
#define ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_BI_H
#include <iosfwd>

class BiList;
class Iterator;

class BiNode {
    friend BiList;
    friend Iterator;
    BiNode *prev;
    BiNode *next;
    int data;

    BiNode() { prev = nullptr, next = nullptr, data = 0; }
};

class Iterator {
    BiNode *node;

public:
    Iterator() { node = nullptr; }
    Iterator(BiNode *p) { node = p; }
    Iterator(const Iterator &it) { node = it.node; }
    bool operator==(const Iterator &it) { return node == it.node; }
    bool operator!=(const Iterator &it) { return node != it.node; };

    Iterator &operator++;
    Iterator &operator--;
    Iterator &operator+(int);
    Iterator &operator-(int);

    int& operator*() { return node->data; }
    int value() { return node->data; }
    void setValue(int i) { node->data = i; }
};

class BiList {
    BiNode *head;
    BiNode *tail;
    int size;

public:
```

```

BiList();
BiList(int);
BiList(BiList &);
~BiList() { clear(); }

void pushBack(int);
void popBack();
void pushFront(int);
void popFront();
void insert(int, int);
void remove(int);
int find(int &);
void clear();

int &operator[](int);
BiList &operator=(BiList &);
int operator()() { return size; }
BiList operator*(BiList &);

friend std::ostream &operator<<(std::ostream &, BiList);
friend std::istream &operator>>(std::istream &, BiList &);

Iterator begin() { return Iterator(head); }
Iterator end() { return Iterator(tail->next); }
};

#endif ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_BI_H

```

KOД Bidirectional.cpp

```

#include "bidirectional.h"
#include <iostream>
#include <fstream>

void BiList::pushBack(int data) {
    auto node = new BiNode;
    node->data = data;
    if (size == 0) {
        head = node;
        tail = node;
        size++;
        return;
    }
}

```

```

    }

    node->prev = tail;
    tail->next = node;
    tail = node;
    size++;
}

void BiList::popBack() {
    if (size == 0)
        return;

    auto del = tail;
    tail = tail->prev;

    if (tail != nullptr)
        tail->next = nullptr;
    else
        head = nullptr;

    delete del;
    size--;

}

```

```

void BiList::pushFront(int data) {
    auto node = new BiNode;
    node->data = data;
    if (size == 0) {
        head = node;
        tail = node;
        size++;
        return;
    }

    node->next = head;
    head->prev = node;
    head = node;
    size++;

}

```

```

void BiList::popFront() {
    if (size == 0)
        return;

    auto del = head;
    head = head->next;

    if (head != nullptr)
        head->prev = nullptr;
    else
        tail = nullptr;

    delete del;
    size--;
}

```

```

void BiList::insert(int idx, int data) {
    if (idx == size) {
        pushBack(data);
        return;
    }
    if (idx == 0) {
        pushFront(data);
        return;
    }

    if (!(0 < idx && idx < size)) {
        return;
    }

    auto node = head;
    for (int i = 0; i < idx; i++) {
        node = node->next;
    }

    auto newNode = new BiNode;
    newNode->data = data;

    node->prev->next = newNode;
    newNode->prev = node->prev;
}

```

```

node->prev = newNode;
newNode->next = node;

size++;

}

void BiList::remove(int idx) {
    if (!(0 <= idx && idx < size)) {
        return;
    }
    if (idx == 0) {
        popFront();
        return;
    }
    if (idx == size - 1) {
        popBack();
        return;
    }

    auto node = head;
    for (int i = 0; i < idx; i++) {
        node = node->next;
    }

    node->next->prev = node->prev;
    node->prev->next = node->next;

    delete node;
    size--;

}

int BiList::find(int &data) {
    auto node = head;
    int i = 0;
    while (node != nullptr) {
        if (node->data == data) {
            return i;
        }
    }
}

```

```

        i++;
        node = node->next;
    }

    return -1;
}

void BiList::clear() {
    auto node = head;
    while (node != nullptr) {
        auto tmp = node->next;
        delete node;
        node = tmp;
    }
    head = nullptr;
    tail = nullptr;
    size = 0;

}

int &BiList::operator[](int idx) {
    if (!0 <= idx && idx < size) {
        exit(1);
    }

    auto node = head;
    for (int i = 0; i < idx; i++) {
        node = node->next;
    }

    return node->data;
}

BiList &BiList::operator=(BiList &l) {
    if (this == &l)
        return *this;

    this->clear();
    for (int i = 0; i < l.size; i++) {
        this->pushBack(l[i]);
    }
}

```



```

    return *this;
}

BiList BiList::operator*(BiList &l) {
    BiList res;
    int range = std::min(l.size, this->size);
    for (int i = 0; i < range; i++) {
        res.pushBack((*this)[i] * l[i]);
    }
    return res;
}

BiList::BiList() {
    head = nullptr;
    tail = nullptr;
    size = 0;
}

BiList::BiList(int s) : BiList() {
    for (int i = 0; i < s; i++)
        pushBack(0);
}

BiList::BiList(BiList &l) : BiList() {
    for (int i = 0; i < l.size; i++) {
        pushBack(l[i]);
    }
}

std::ostream &operator<<(std::ostream &stream, BiList l) {
    for (int i = 0; i < l.size; i++)
        stream << l[i] << 't';
    stream << '\n';
    return stream;
}

std::istream &operator>>(std::istream &stream, BiList &l) {
    std::cout << "Size? ";
    int s;
    stream >> s;
    std::cout << "Elements? ";
    for (int i = 0; i < s; i++) {
        int a;

```

```

        stream >> a;
        l.pushBack(a);
    }
    return stream;
}

Iterator & Iterator::operator+(int a) {
    for (int i = 0; i < a; i++)
        node = node->next;
    return *this;
}

Iterator & Iterator::operator-(int a) {
    for (int i = 0; i < a; i++)
        node = node->prev;
    return *this;
}

Iterator & Iterator::operator--() {
    node = node->prev;
    return *this;
}

Iterator & Iterator::operator++() {
    node = node->next;
    return *this;
}

```

КОД main.cpp

```

#include <iostream>
#include "bidirectional.h"

using namespace std;

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    for (int l : arr)
        cout << l << ' ';
    BiList list1, list2;
    cin >> list1 >> list2;
    BiList list3 = list2;

    for (int i = 0; i < list2(); i++)
        list2[i] = -1;
}

```

```

cout << list2;

cout << list1 * list3;

for (auto i : list1) {
    cout << i << ' ';
}
cout << '\n';
cout << '\n';

auto iter = list1.begin();
cout << (iter + 2).value() << ' ';
cout << (--iter).value() << ' ';
cout << (iter - 1).value() << ' ';
cout << '\n';

iter = list2.begin();
iter + 2;
iter.setValue(10);
cout << list2;
}

```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

```

1 2 3 4 5 Size? 5
Elements? 6 7 8 9 10

Size? 3
Elements? 1 2 3

-1      -1      -1
6       14      24

6:7:8:9:10:

8 7 6

-1      -1      10

```

ВОПРОСЫ

1. Что такое абстрактный тип данных? Привести примеры АТД

АТД — тип данных, определяемый только через операции, которые могут выполняться над соответствующими объектами безотносительно к способу представления этих объектов.

Пример АТД: Примером абстрактного типа данных является класс в языке C++.

2. Привести примеры абстракции через параметризацию.

Абстракция через параметризацию может быть осуществлена так же, как и для функций

3. Привести примеры абстракции через спецификацию.

Абстракция через спецификацию достигается за счёт того, что операции представляются как часть типа.

4. Что такое контейнер? Привести примеры.

Контейнер — набор однотипных элементов.

Примеры: массив, список, стек, очередь.

5. Какие группы операций выделяют в контейнерах?

- Операции доступа к элементам, которые обеспечивают и операцию замены значений элементов;
- Операции добавления и удаления элементов или групп элементов;
- Операции поиска элементов и групп элементов;
- Операции объединения контейнеров;
- Специальные операции, которые зависят от вида контейнера.

6. Какие виды доступа к элементам контейнера существуют? Привести примеры.

Доступ к элементам контейнера бывает: последовательный, прямой и ассоциативный.

- Прямой: Прямой доступ — это доступ по индексу. Например, `a[10]` — требуется найти элемент контейнера с номером 10.

- Ассоциативный: Ассоциативный доступ также выполняется по индексу, но индексом будет являться не номер элемента, а его содержимое. Пример: `a["word"]` — словарь с парами слово — перевод.

- Последовательный: При последовательном доступе осуществляется перемещение от элемента к элементу контейнера.

7. Что такое итератор?

Итератор — это объект, который обеспечивает последовательный доступ к элементам контейнера.

8. Каким образом может быть реализован итератор?

Итератор может быть реализован как часть класса-контейнера в виде набора методов (`v.first()`, `v.last()`, `v.next()`, `v.prev()`, `v.skip(n)`, `v.current()`).

Итератор можно реализовать как класс, представляющий такой же набор операций.

В С++ итератор реализуется как класс, который имеет такой же интерфейс, как и указатель для совместимости с массивами. (iterv++, iterv--, *iterv и т.д.)

9. Каким образом можно организовать объединение контейнеров?

- Простое сцепление двух контейнеров: в новый контейнер попадают сначала элементы первого контейнера, потом второго, операция не коммутативна.
- Объединение упорядоченных контейнеров, новый контейнер тоже будет упорядочен, операция коммутативна.
- Объединение контейнеров как объединение множеств, в новый контейнер попадают только те элементы, которые есть хотя бы в одном контейнере, операция коммутативна.
- Объединение контейнеров как пересечение множеств, в новый контейнер попадают только те элементы, которые есть в обоих контейнерах, операция коммутативна.
- Операция вычитания, в контейнер попадают только те элементы первого контейнера, которых нет во втором, операция не коммутативна.
- Извлечение части элементов из контейнера и создание нового контейнера... часть контейнера задаётся двумя итераторами.

10. Какой доступ к элементам предоставляет контейнер, состоящий из элементов «ключ-значение»?

Контейнер map, который представляет ассоциативный доступ, состоит из пар "ключ-значение". — то есть ассоциативный доступ.

11. Как называется контейнер, в котором вставка и удаление элементов выполняется на одном конце контейнера?

Если вставка и удаление осуществляется на одном конце, то такой контейнер называется стеком (Last In First Out — последним пришёл, первым ушёл).

12. Какой из объектов (a,b,c,d) является контейнером?

- a. int mas=10;
- b. 2. int mas;
- c. 3. struct {char name[30]; int age;} mas;
- d. 4. int mas[100];

Ответ: d

13. Какой из объектов (a,b,c,d) не является контейнером?

- a. int a[]={1,2,3,4,5};
- b. 2. int mas[30];
- c. 3. struct {char name[30]; int age;} mas[30];
- d. 4. int mas;

Ответ: d

14. Контейнер реализован как динамический массив, в нем определена операция доступ по индексу. Каким будет доступ к элементам контейнера?

Прямой доступ — это доступ по индексу, следовательно, доступ будет прямым.

15. Контейнер реализован как линейный список. Каким будет доступ к элементам контейнера?

Линейный список не поддерживает доступ по индексу, поэтому доступ будет последовательным: при последовательном доступе осуществляется перемещение от элемента к элементу контейнера.